

Java 컬렉션(List, Map, Set) – 언제, 왜 어떤 것을 쓰는지

List 순서가 있고 중복을 허용한다.

List 를 출력하기 위해서는 List.get(0) ~~를 사용하면 된다

Set은 중복을 허용하지 않는다

Set의 크기를 알려면 Set.size()를 사용하면 된다.

Set은 순서가 없는 주머니 같은 자료구조이기 때문에 0번째,1번째 개념이 없다

그래서 반복자 Iterator가 필요하다.

Map : key와 value의 쌍으로 이루어져있습니다. Key는 중복이 불가하고, Value는 중복이 가능합니다.

ArrayList vs LinkedList, HashMap / TreeMap / LinkedHashMap 개념 정리

ArrayList : 내부적으로 배열(Array)을 사용하여 데이터를 관리합니다. 배열의 크기를 동적으로 늘려줍니다.

LinkedList : 데이터와 다음/이전 데이터의 주소(참조)를 묶은 노드들을 선으로 연결하여 관리합니다. (이중 연결 리스트 구조)

ArrayList는 인덱스로 바로 접근이 가능해서 데이터 조회가 빠르다.

데이터 추가 삭제 시 느리다.

이유는 빈자리를 채우기 위해 데이터를 밀거나 당겨야 해서

LinkedList는 앞뒤 노드의 연결(링크)만 바꿔주면 돼서 빠르다.

HashMap : 해시 함수를 이용해 데이터를 해시 테이블에 저장합니다.

TreeMap : 이진 탐색 트리의 일종인 레드-블랙 트리(Red-Black Tree) 기반으로 데이터를 저장합니다.

LinkedHashMap: HashMap과 동일하지만, 내부에 연결 리스트를 추가로 두어 데이터가 들어온 순서를 기억합니다.

기본적으로는 HashMap을 사용

이유는 특별한 정렬이나 순서가 필요 없다면 압도적으로 성능이 좋다.

TreeMap은 데이터가 정렬된 상태로 유지 되어야함

LinkedHashMap은 데이터가 입력된 순서가 중요할 때 사용합니다.

예) 화면에 입력된 순서대로 목록을 보여줘야할 때

자습 과제 1 : 실무 시나리오 3~4개에서 어떤 컬렉션을 쓸 것인지를 선택하고,

왜 그런지 이유를 서술

1. 실무 시나리오 : 수강생 1만 명의 수강 이력에서 중복 없이 과목 목록을 뽑아야 한다면?

답변 : 일단 Set 컬렉션을 활용해서 중복을 없애고 사용할 거 같습니다.

2. 실무 시나리오 : 수강생 이름으로 빠르게 검색해야 한다면?

답변 :

Map 컬렉션을 활용하겠습니다.

수강생의 학번을 key 값으로 하고 value에는 이름을 두겠습니다.

3. 실무 시나리오 : 선생님들의 입사년도에 따라 구분을 해서 순서로 뽑을 때 사용하는 것은?

답변 : List 컬렉션을 사용하겠습니다.

이유는 입사년도 순으로 순서대로 넣기 때문에 순서가 있는 List를 사용하면 될 거 같습니다.

4. 실무 시나리오 : 접수를 받을 당시에 학생들을 선착순으로 했을 때 사용해야 하는 컬렉션은?

답변 : LinkedList를 사용하겠습니다.

FIFO 원칙이 중요하므로 Queue 인터페이스를 구현한 LinkedList 컬렉션을 사용하겠습니다. 추가 와 삭제가 빠르기 때문에 선착순에 활용 가능 할거 같습니다!

자습 과제 2 : MariaDB 10만건 이상 생성

피드백 전

루틴 본문:

```
1 BEGIN
2   DECLARE i INT;
3   SELECT COUNT(*) INTO i FROM test.userinfo;
4   WHILE(i<100000) DO
5       INSERT INTO test.userinfo(NAME,CLASS)VALUES('정소원','창조반');
6       SET i = i + 1;
7   END WHILE;
8 END
```

10만건 생성 프로시저

피드백 적용 후

루틴 본문:

```
1 BEGIN
2   DECLARE i INT;
3   SELECT COUNT(*) INTO i FROM test.userinfo;
4   SET autocommit = 0;
5   START TRANSACTION;
6   WHILE(i<100000) DO
7       INSERT INTO test.userinfo(NAME,CLASS)VALUES(CONCAT('user',i), FLOOR(RAND()*100));
8       SET i = i + 1;
9   END WHILE;
10  COMMIT;
12 END
```

User에 숫자를 붙여서 이름을 구분 지었고, FLOOR함수를 활용해서 랜덤으로 CLASS가 생기게 하였습니다.

```
CALL datainsert();
#DELETE FROM test.userinfo;
/* 영향 받은 행: 100,001  찾은 행: 0  경고: 0  지속 시간 2 쿼리: 0.985 초 */
SELECT `ID`, `USER`, `HOST`, `DB`, `COMMAND`, `TIME`, `STATE`, LEFT(`INFO`, 51200) AS `Info`, `TIME`
```

START TRANSACTION을 사용하니 화면에서의 버퍼가 보이지 않았고,

확실하게 빨라졌습니다.

피드백 전

test.userinfo: 100,100 행 (총) (대략적), 제한 수: 1,000

#	1 No	2 NAME	3 CLASS
1	100,000	정소원	창조반
2	99,999	정소원	창조반
3	99,998	정소원	창조반
4	99,997	정소원	창조반
5	99,996	정소원	창조반
6	99,995	정소원	창조반
7	99,994	정소원	창조반
8	99,993	정소원	창조반
9	99,992	정소원	창조반
10	99,991	정소원	창조반
11	99,990	정소원	창조반
12	99,989	정소원	창조반
13	99,988	정소원	창조반
14	99,987	정소원	창조반
15	99,986	정소원	창조반
16	99,985	정소원	창조반
17	99,984	정소원	창조반
18	99,983	정소원	창조반
19	99,982	정소원	창조반
20	99,981	정소원	창조반
21	99,980	정소원	창조반

10만건 생성 결과

피드백 반영 후

test.userinfo: 100,356 행 (총) (대략적), 제한 수: 1,000

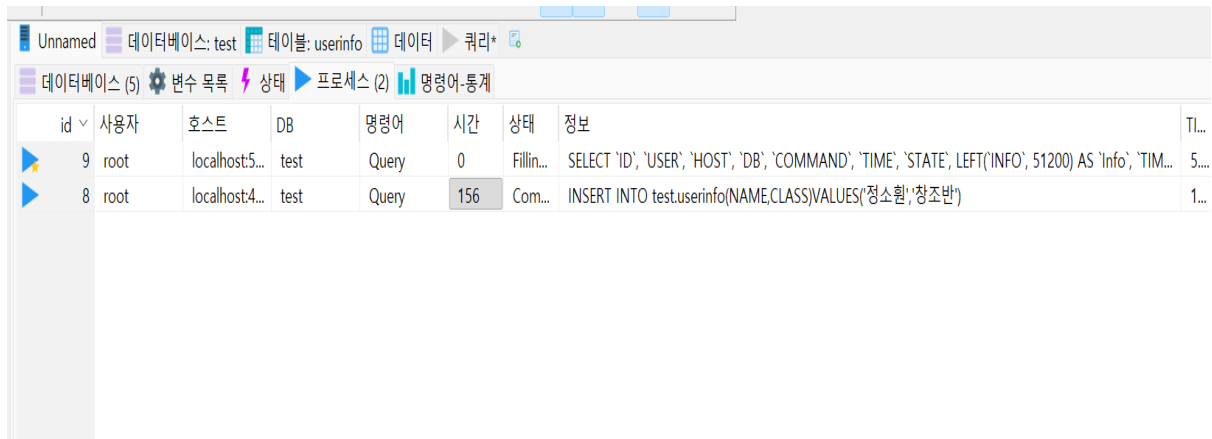
#	1 No	2 NAME	3 CLASS
1	100,000	user99999	19
2	99,999	user99998	98
3	99,998	user99997	90
4	99,997	user99996	52
5	99,996	user99995	56
6	99,995	user99994	43
7	99,994	user99993	87
8	99,993	user99992	30
9	99,992	user99991	88
10	99,991	user99990	71
11	99,990	user99989	55
12	99,989	user99988	69
13	99,988	user99987	96
14	99,987	user99986	4
15	99,986	user99985	42
16	99,985	user99984	69
17	99,984	user99983	1
18	99,983	user99982	46
19	99,982	user99981	42
20	99,981	user99980	89
21	99,980	user99979	34

10만건 생성 결과

*** 이부분은 트랜잭션을 적용하니 해결 됐습니다.***

여기서 살짝 이슈가 있다면 프로시저를 돌린 상태에서 하이드가 먹통이 되는 경우가 있습니다!

그래서 하이드를 작업관리자에서 종료하고 다시 켜서 확인해보고 있습니다!



id	사용자	호스트	DB	명령어	시간	상태	정보	TI...
9	root	localhost:5...	test	Query	0	Fillin...	SELECT 'ID', 'USER', 'HOST', 'DB', 'COMMAND', 'TIME', 'STATE', LEFT('INFO', 51200) AS 'Info', 'TIM...	5...
8	root	localhost:4...	test	Query	156	Com...	INSERT INTO test.userinfo(NAME,CLASS)VALUES('정소환','창조반')	1...

꺼진 하이드를 다시 켜서 프로세스 확인